

BestDevOps Pulse

World-Class MVP Project Blueprint

Delivery, Reliability, and SLO Intelligence for Software Teams

Mission

Help engineering leaders know which services are shipping fast, which are breaking, which are burning reliability budget, and what should be fixed next.

MVP Scope

Only 3 pillars: Delivery Performance Intelligence, Reliability Intelligence, and SLI/SLO/Error Budget Intelligence. Everything else is explicitly out of v1.

Prepared for

BestDevOps.com / SCMGalaxy ecosystem - July 2026

Version 1.0 | Founder + Architect + Designer + Developer + QA + DevOps + SRE Blueprint

Table of Contents

- 1. Executive Summary
- 2. Product Thesis and Positioning
- 3. MVP Scope and Non-Goals
- 4. Target Users and Buyer Narrative
- 5. Product Information Architecture
- 6. Core User Journeys
- 7. High-Level Architecture
- 8. Low-Level Architecture
- 9. Data Model and Storage Design
- 10. Integration Design
- 11. Metric Calculation Engine
- 12. Scoring and Data Confidence Model
- 13. Dashboards and UX Specification
- 14. Reporting and Recommendations
- 15. Security, Privacy, and Governance
- 16. API Specification
- 17. Technology Stack
- 18. DevOps and Infrastructure Plan
- 19. SRE Plan for BestDevOps Pulse
- 20. QA and Testing Strategy
- 21. Implementation Roadmap
- 22. MVP Backlog
- 23. Risks and Mitigations
- 24. Launch and Pricing Plan
- 25. Future Roadmap
- 26. Glossary
- 27. Reference Sources

1. Executive Summary

BestDevOps Pulse is a focused SaaS MVP for engineering leaders and platform/SRE teams. It connects metadata from source control, CI/CD, incidents, and observability tools to produce service-level delivery and reliability intelligence.

The MVP deliberately avoids broad Engineering Intelligence, FinOps, AI readiness, DevSecOps, vendor comparison, and learning workflows. Those can become future modules, but including them now will create scope explosion and delay sellable delivery.

Single product promise

BestDevOps Pulse tells you whether each service is fast, reliable, and safe to change - using DORA, SRE, and SLI/SLO/Error Budget practices.

MVP Pillar	Framework Basis	Business Question	Primary Output
Delivery Performance Intelligence	DORA Metrics [R1]	Are we shipping fast and safely?	Deployment frequency, lead time, change failure rate, recovery time, rework rate
Reliability Intelligence	Google SRE practices [R2]	Are services operationally healthy?	Incident trend, MTTA, MTTR, repeat incidents, alert noise, postmortem coverage
SLO/Error Budget Intelligence	SLI/SLO/Error Budget [R2][R3], OpenSLO later [R4]	Are we meeting reliability promises?	SLO compliance, error budget remaining, burn rate, breach risk

The strongest differentiator is not the dashboard itself. The differentiator is the Service Reliability Timeline: a unified chronological view that connects deployments, incidents, rollbacks, SLO burn, and recovery events for each service.

2. Product Thesis and Positioning

BestDevOps.com should not become another tutorial portal, DevOps OS, or generic metrics dashboard. The market already has DORA dashboards and observability platforms. The gap is a service-level intelligence layer that connects delivery events to reliability outcomes.

Positioning statement

BestDevOps Pulse connects delivery, incident, and SLO data to show which services are improving, which are breaking, and which are burning reliability budget.

Do Not Position As	Reason	Position As Instead
DORA dashboard	Crowded and already present in GitLab, Harness, LinearB, etc.	Delivery + Reliability Intelligence
Observability platform	Datadog, Grafana, New Relic, Prometheus ecosystem already dominate	Observability consumer and executive insight layer
Incident tool	PagerDuty/Opsgenie/ServiceNow already own incident operations	Incident-performance correlation layer
SLO tool only	Nobl9, Datadog, Grafana Cloud already support SLO management	SLO plus release-risk and service timeline layer
SCMGalaxy OS	That is learning/building/execution-oriented	Measurement and reliability intelligence for leaders

3. MVP Scope and Non-Goals

MVP scope must be aggressively limited. The first version should prove one thesis: engineering leaders will pay for a trustworthy view of service-level delivery speed, production reliability, and SLO health.

Included in MVP	Excluded from MVP
Organization, Project, Service, Environment model	Cloud cost optimization / FinOps
Read-only integrations	AI adoption scoring
GitHub/GitLab + GitHub Actions/Jenkins	DevSecOps/compliance framework scoring
Jira/manual incidents	Vendor directory/decision platform
Prometheus/Datadog SLO data or manual SLO entry	Learning, labs, courses, certifications
DORA dashboard	Full source code analysis
Reliability dashboard	Deep log ingestion / APM replacement
SLO/Error Budget dashboard	ChatOps assistant
Service Reliability Timeline	AI-generated code fixes
Monthly PDF report	Enterprise SSO/SCIM in v1

MVP success metric

A pilot customer connects 3-10 services and receives a monthly report that identifies at least 3 useful delivery or reliability insights they did not have in one place before.

4. Target Users and Buyer Narrative

Persona	Pain	What They Need	MVP Value
CTO / VP Engineering	Cannot see service health across teams	Executive view of speed, reliability, risk	Org and project scorecards + monthly report
Head of Platform Engineering	Platform work is hard to justify	Evidence of delivery friction and service reliability	Service-level DORA and incident correlations
SRE Manager	Incidents and SLOs are disconnected from releases	Timeline connecting deploys, alerts, and SLO burn	Service Reliability Timeline
Engineering Manager	Team has many tools but poor visibility	Practical trends and improvement targets	Historical views and recommendations
Founder / Small SaaS CTO	Needs maturity without heavy tooling	Simple health audit and roadmap	Lightweight setup, manual + integration mode

Easy sales story

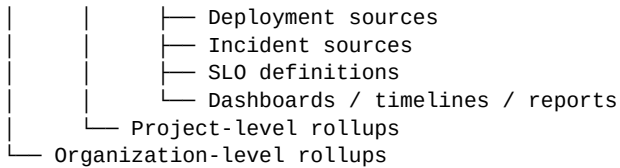
Connect your tools once. Map your services. BestDevOps Pulse shows how fast each service ships, how reliable it is, whether it is burning SLO budget, and what changed over time.

5. Product Information Architecture

The product hierarchy should be service-centric. Project-level dashboards are useful, but DORA, SRE, and SLO metrics become most actionable at service/application level.

```

Organization
├── Members and roles
├── Integrations configured once
├── Projects / Products
│   ├── Services / Applications
│   │   ├── Repositories
│   │   ├── Pipelines
│   │   └── Environments
│   └── ...
└── ...
  
```



Object	Definition	MVP Notes
Organization	Tenant/customer account	Billing, members, integrations, audit logs
Project	Business/product grouping	Motoshare, WizBrand, DevOpsSchool
Service	Deployable or user-facing component	Payment API, Booking API, Admin Portal
Environment	Runtime target	prod, staging, dev; MVP primarily focuses on prod
Integration	External data source	GitHub, GitLab, Jenkins, Jira, Prometheus, Datadog
Mapping	How external entities connect to services	Repo -> Service, Pipeline -> Service, Incident component -> Service
Metric Snapshot	Aggregated metric for period	Daily/weekly/monthly rollups
Timeline Event	Unified service event	Deployment, incident, rollback, SLO burn, alert

6. Core User Journeys

Journey A - Founder/CTO creates first organization

1. User signs up with email/social login.
2. Creates organization name, slug, timezone, primary industry.
3. Chooses MVP mode: manual audit, integrations, or hybrid.
4. Invites admin/platform/SRE members.
5. Lands on setup checklist.

Journey B - Connect tools once at organization level

6. Admin opens Integrations.
7. Selects GitHub/GitLab, Jira, CI/CD, incident, observability integrations.
8. Grants read-only permissions.
9. BestDevOps runs a connection test and permission audit.
10. Backfill job starts and shows data availability estimate.

Journey C - Create project and map services

11. User creates Project.
12. System suggests services from repositories, pipeline names, Kubernetes namespaces, PagerDuty services, Prometheus labels.
13. User confirms or edits service mapping.
14. Service mapping confidence is shown.
15. Dashboards become available after first aggregation.

Journey D - View dashboards

16. User opens Organization dashboard for rollup.
17. Drills into Project dashboard.
18. Drills into Service dashboard.
19. Changes time range: week, month, 3 months, 6 months, 1 year, custom.
20. Opens Service Reliability Timeline for details.

Journey E - Monthly report

21. On first day of month, scheduled report job generates PDF.
22. Report includes delivery, reliability, SLO health, risks, and recommendations.
23. User downloads report or sends to team.
24. Previous reports remain archived.

7. High-Level Architecture

The MVP should use a modular monolith API plus asynchronous workers. This avoids premature microservices while keeping boundaries clean enough to split later.



Component	Responsibility	MVP Technology
Web App	UI, dashboards, onboarding, settings	Next.js + TypeScript + Tailwind + shadcn/ui
API Backend	Business logic, REST APIs, auth enforcement	FastAPI + Python 3.12
Worker	Integrations, backfill, aggregation, reports	Celery/RQ + Redis
Database	Tenant data, events, metrics	PostgreSQL 16; TimescaleDB optional
Cache/Queue	Job queue, locks, rate-limit state	Redis
Object Storage	Reports, exports, logs if needed	S3/MinIO
Scheduler	Periodic ingestion and aggregation	Celery Beat/APScheduler
Observability	App metrics/logs/traces	Prometheus + Grafana + OpenTelemetry

8. Low-Level Architecture

8.1 Backend modules

Module	Responsibilities	Important Classes/Services
Identity & Tenancy	Users, orgs, roles, memberships, invite flow	AuthService, RBACService, TenantContext
Org/Project/Service	Domain model and service mapping	OrgService, ProjectService, ServiceCatalogService, MappingService
Integration Hub	OAuth/app tokens, permissions, sync status	IntegrationService, CredentialVault, ConnectorRegistry
Connector Workers	Pull/webhook data from external tools	GitHubConnector, GitLabConnector, JenkinsConnector, JiraConnector, PrometheusConnector
Event Normalizer	Convert tool-specific payloads to canonical events	EventNormalizer, EventDeduplicator, EventRouter
Metric Engine	Calculate DORA/SRE/SLO metrics	DoraCalculator, ReliabilityCalculator, SloCalculator
Scoring Engine	Score and data confidence	ScoreService, ConfidenceService
Timeline Engine	Build unified service timeline	TimelineService
Reporting	Monthly PDF and export generation	ReportService, ReportRenderer
Audit & Security	Audit logs, secrets access, integration changes	AuditLogService, PermissionAuditService

9. Data Model and Storage Design

The MVP database must handle relational tenant data plus time-series event data. Use PostgreSQL as the primary system of record. Add TimescaleDB only if time-series volume grows, but design tables so migration is easy.

9.1 Core entity relationship

```

users --< org_members >-- organizations --< projects --< services --< environments
organizations --< integrations --< integration_credentials
services --< service_mappings >-- external_entities
services --< deployments
services --< incidents
services --< slo_definitions --< sli_measurements --< error_budget_snapshots
services --< timeline_events
services --< metric_snapshots
organizations --< reports

```

9.2 Required MVP tables

Table	Purpose	Key Columns
users	User identity	id, email, name, status, created_at
organizations	Tenant account	id, name, slug, timezone, plan, created_by, created_at
org_members	Membership and RBAC	org_id, user_id, role, status
projects	Product/project grouping	id, org_id, name, key, status
services	Main metric unit	id, org_id, project_id, name, type, criticality, owner_team
environments	Runtime environments	id, service_id, name, type, is_production
integrations	External tool connections	id, org_id, provider, status, last_sync_at
integration_credentials	Encrypted credentials metadata	id, integration_id, secret_ref, expires_at
external_entities	Tool-side resources	id, integration_id, entity_type, external_id, name
service_mappings	Entity-to-service mapping	service_id, external_entity_id, confidence, mapping_rule
deployments	Normalized deployments	id, service_id, environment_id, version, status, started_at, finished_at

Table	Purpose	Key Columns
changes	Commit/PR/change units	id, service_id, repo_id, commit_sha, pr_id, merged_at
incidents	Normalized incidents	id, service_id, severity, status, detected_at, acknowledged_at, resolved_at
alerts	Alert events	id, service_id, source, status, fired_at, resolved_at
slo_definitions	SLO targets	id, service_id, name, sli_type, target, window_days
sli_measurements	Raw/aggregated SLI values	id, slo_id, period_start, period_end, good_events, total_events
error_budget_snapshots	Computed budget state	id, slo_id, period_start, budget_remaining_pct, burn_rate
metric_snapshots	Daily/weekly/monthly rollups	id, service_id, metric_family, granularity, period_start, values_json
timeline_events	Unified service timeline	id, service_id, event_type, event_time, severity, payload_json
reports	Monthly/audit reports	id, org_id, report_type, period_start, file_url
audit_logs	Security and admin events	id, org_id, actor_id, action, target_type, created_at

9.3 Sample table DDL snippets

```
CREATE TABLE services (
  id UUID PRIMARY KEY,
  org_id UUID NOT NULL REFERENCES organizations(id),
  project_id UUID NOT NULL REFERENCES projects(id),
  name TEXT NOT NULL,
  service_key TEXT NOT NULL,
  type TEXT NOT NULL CHECK (type IN ('api', 'web', 'worker', 'mobile', 'library', 'infra')),
  criticality TEXT NOT NULL DEFAULT 'medium',
  owner_team TEXT,
  lifecycle_status TEXT NOT NULL DEFAULT 'active',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE(org_id, service_key)
);
```

```
CREATE TABLE timeline_events (
  id UUID PRIMARY KEY,
  org_id UUID NOT NULL,
  project_id UUID NOT NULL,
  service_id UUID NOT NULL REFERENCES services(id),
  event_type TEXT NOT NULL,
  event_time TIMESTAMPTZ NOT NULL,
  severity TEXT,
  source_tool TEXT NOT NULL,
  source_event_id TEXT,
  payload_json JSONB NOT NULL DEFAULT '{}::jsonb',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE(org_id, source_tool, source_event_id)
);
```

```
CREATE INDEX idx_timeline_service_time ON timeline_events(service_id, event_time DESC);
CREATE INDEX idx_timeline_org_time ON timeline_events(org_id, event_time DESC);
CREATE INDEX idx_timeline_payload_gin ON timeline_events USING GIN(payload_json);
```

10. Integration Design

Integrations should be configured once at organization level and mapped to services. Every integration must support permission test, backfill, incremental sync, webhook when available, retry, rate-limit handling, and disable/delete.

Integration	MVP Data	Ingestion Mode	Notes
GitHub	Repos, PRs, commits, releases, workflow runs	API + webhooks	Use fine-grained permissions where possible [R5]
GitLab	Projects, MRs, pipelines, environments	API + webhooks	Alternative to GitHub
GitHub Actions	Workflow runs, jobs, durations, failures	GitHub API/webhooks	Often easiest v1 CI source
Jenkins	Jobs, builds, stages, statuses	Remote API polling	Jenkins exposes REST-like remote APIs [R7]
Jira	Incidents, bugs, postmortems, change tickets	API + webhooks	Jira webhooks notify apps on issue events [R6]
Prometheus	SLI queries, availability/latency/error rate	HTTP API query_range	Prometheus query_range supports time windows [R8]
Datadog	SLOs, service monitors, incidents optional	API	Datadog provides SLO APIs [R9]
Manual Upload	Deployments/incidents/SLOs CSV	CSV/form	Critical for MVP when integrations are unavailable

10.1 Connector lifecycle

1. Register integration provider.
2. Complete OAuth/token setup.
3. Store encrypted credential reference.
4. Run permission test.
5. Discover external entities.
6. Suggest service mappings.
7. Run historical backfill.
8. Start incremental sync.
9. Process webhooks when available.
10. Aggregate metrics daily and on demand.

10.2 Connector interface contract

```
class BaseConnector:
    provider: str

    def test_connection(self, credentials) -> ConnectionResult: ...
    def discover_entities(self, org_id) -> list[ExternalEntity]: ...
    def backfill(self, integration_id, start_at, end_at) -> BackfillResult: ...
    def incremental_sync(self, integration_id, cursor) -> SyncResult: ...
    def normalize_event(self, raw_event) -> list[CanonicalEvent]: ...
    def handle_webhook(self, payload, headers) -> WebhookResult: ...
```

11. Metric Calculation Engine

11.1 DORA metrics

DORA metrics form the delivery performance layer. The product should cite DORA but not stop at DORA; it must add service mapping, confidence, timeline, and recommendations.

Metric	Formula / Logic	Primary Sources	Confidence Dependencies
Deployment Frequency	Count successful production deployments in period	CI/CD, deployment events, Kubernetes/ArgoCD	Production environment mapping
Lead Time for Changes	prod_deployed_at - commit_created_at or PR_merged_at	Git + CI/CD deployments	Change-to-deployment linking
Change Failure Rate	failed_changes / total_deployments	Deployments + incidents + rollbacks	Failure tagging and incident correlation
Failed Deployment Recovery Time	resolved_at - failure_started_at	Incidents, rollback events	Incident status timestamps
Deployment Rework Rate	rework_deployments /	Rollback, revert, hotfix labels	Rework detection rules

Metric	Formula / Logic	Primary Sources	Confidence Dependencies
	total_deployments		

DORA calculation window examples:

- weekly: Monday 00:00 to Sunday 23:59 in org timezone
- monthly: calendar month in org timezone
- custom: user-selected UTC-normalized range

Production deployments only:

- environment.is_production = true
- or deployment target name matches configured prod aliases
- or service mapping explicitly marks pipeline as production deployer

11.2 Reliability metrics

Metric	Definition	Data Needed
Incident Count	Number of incidents in period by severity	Incident records, severity, service mapping
MTTA	acknowledged_at - detected_at	Incident detection and ack timestamps
MTRR	resolved_at - detected_at	Incident detection and resolve timestamps
Repeat Incidents	Same service/root-cause/signature recurring within configured window	Root cause/category/component tags
Postmortem Coverage	incidents_with_postmortem / eligible_incidents	Postmortem links/status
Alert Noise	non_actionable_alerts / total_alerts	Alert and incident linking
Reliability Trend	Composite score change over time	Historical metric snapshots

11.3 SLI/SLO/Error Budget metrics

Concept	Definition	MVP Implementation
SLI	Quantitative measure of service level [R2]	Availability, latency, error rate, throughput
SLO	Target value/range measured by SLI [R2]	Manual form first; OpenSLO import later [R4]
Error Budget	Allowed unreliability; 1 - SLO target [R3]	Calculated per SLO window
Burn Rate	How fast budget is being consumed	Observed bad-event rate / allowed bad-event rate
SLO Breach Risk	Likelihood budget will be exhausted before window end	Burn rate + remaining budget trend

Request-based SLO example:

```
target = 99.9%
total_events = 1,000,000
bad_events_allowed = total_events * (1 - 0.999) = 1,000
actual_bad_events = total_events - good_events
budget_consumed_pct = actual_bad_events / bad_events_allowed * 100
budget_remaining_pct = max(0, 100 - budget_consumed_pct)
```

Burn rate:

```
allowed_error_rate = 1 - target
observed_error_rate = bad_events / total_events
burn_rate = observed_error_rate / allowed_error_rate
```

12. Scoring and Data Confidence Model

Scores make the dashboard executive-friendly, but raw metrics must remain visible. A score without explainability will not earn trust.

Score	Weight	Subsignals
Delivery Performance Score	40%	Deployment frequency, lead time, change failure rate, recovery time, rework rate

Score	Weight	Subsignals
Reliability Score	35%	Incident severity, MTTA, MTTR, repeats, postmortem coverage, alert noise
SLO Health Score	25%	SLO coverage, compliance, budget remaining, burn rate, breach count

BestDevOps Pulse Score =
 0.40 * DeliveryPerformanceScore +
 0.35 * ReliabilityScore +
 0.25 * SloHealthScore

Do not hide raw metrics. Show score -> contributing factors -> raw events.

12.1 Data Confidence Score

Data confidence is mandatory because engineering telemetry is messy. It prevents false precision and helps users trust the platform.

Confidence Factor	Why It Matters	Example
Source Coverage	Are required tools connected?	GitHub connected but CI/CD missing -> lead time confidence medium
Mapping Completeness	Are repos/pipelines/incidents mapped to services?	80% repos mapped -> high
Backfill Depth	How much historical data is available?	Only 14 days of CI history -> low for 6-month view
Event Consistency	Do timestamps and statuses align?	Deployment finish time missing -> medium
Manual Overrides	Manual data may be useful but less reliable	Manual incident import -> medium

Metric confidence levels:

- High: score >= 80
- Medium: 50-79
- Low: < 50

Every dashboard metric should display confidence: High / Medium / Low.

13. Dashboards and UX Specification

13.1 Navigation

Left Navigation:

- Home
- Organizations
- Projects
- Services
- Dashboards
 - Delivery Performance
 - Reliability
 - SLO / Error Budget
 - Service Timeline
- Reports
- Integrations
- Mapping
- Settings
- Audit Logs

13.2 Org dashboard

Widget	Purpose	MVP Details
Pulse Score	Executive rollout	Delivery, Reliability, SLO scores
Top Risky Services	Focus attention	Services with low score or high burn rate

Widget	Purpose	MVP Details
Trend Selector	Historical view	Week/month/3m/6m/1y/custom
Integration Health	Data trust	Last sync, failures, confidence
Monthly Report CTA	Shareable output	Download latest PDF

13.3 Project dashboard

Widget	Description
Project Score	Rollup from all services in project
Delivery Trends	Deployment frequency, lead time, CFR, recovery time
Reliability Trends	Incident count, MTTA, MTTR, postmortem coverage
SLO Coverage	Services with/without SLOs
Service Table	Sortable table by risk, score, burn rate, incident count

13.4 Service dashboard

Service: Payment API
Time range: Last 30 days

Pulse Score: 67/100
Delivery: 72 | Reliability: 60 | SLO: 69

DORA:

- Deployments: 18
- Lead time: 1.8 days
- Change failure rate: 14%
- Recovery time: 2h 14m

Reliability:

- P1: 1 | P2: 3 | P3: 9
- MTTA: 12m
- MTTR: 2h 14m
- Repeat incidents: 2

SLO:

- Availability target: 99.9%
- Current: 99.82%
- Budget remaining: 18%
- Burn rate: 3.1x

13.5 Service Reliability Timeline

This is the signature feature. It gives a single narrative across deploys, incidents, alerts, and SLO burn.

July 01 10:14 - Deployment v2.4.1 succeeded
 July 01 10:32 - Error rate increased from 0.2% to 1.8%
 July 01 10:38 - SLO burn rate reached 6x
 July 01 10:45 - P1 incident opened
 July 01 11:05 - Rollback deployment started
 July 01 11:28 - Incident resolved
 July 01 11:31 - MTTR recorded: 46 minutes
 July 02 09:00 - Postmortem missing after 24 hours

14. Reporting and Recommendations

The monthly PDF report is critical for sales. Many leaders do not live in dashboards; they share reports in leadership meetings.

Report Section	Content
Executive Summary	Overall score, trend, top wins, top risks
Delivery Performance	DORA metrics and trends

Report Section	Content
Reliability	Incidents, MTTA, MTTR, repeats, postmortems
SLO/Error Budget	SLO coverage, budget remaining, burn risks
Risky Services	Top 5 services needing action
Improvement Actions	Prioritized, owner, effort, expected impact
Data Confidence	Data gaps and recommended integration/mapping fixes

Recommendation format:

- Finding: Payment API error budget is down to 18% with 3.1x burn rate.
- Evidence: 2 incidents occurred within 24h of deployments v2.4.1 and v2.4.2.
- Action: Pause non-critical deploys for 3 days and fix timeout errors in checkout flow.
- Owner: Payment platform team.
- Impact: Reduce SLO breach risk from High to Medium.
- Confidence: Medium.

15. Security, Privacy, and Governance

Security promise

BestDevOps Pulse is a metadata platform. It does not require source code, production logs with PII, secrets, customer data, or database access for MVP.

Area	MVP Requirement
Access model	Read-only integrations by default
Credential storage	Encrypt tokens using cloud KMS or sealed secrets; store only secret reference in DB
RBAC	Org Owner, Admin, Engineer, Viewer
Audit logs	Record integration changes, role changes, report exports, credential rotations
Tenant isolation	Every table includes org_id and API enforces tenant context
Data retention	Default raw event retention 13 months; configurable later
PII minimization	Store user emails only where needed; redact raw payloads
Network security	TLS everywhere; webhook signature validation
Compliance path	SOC 2 readiness later; implement evidence-friendly controls from day one

15.1 RBAC matrix

Action	Owner	Admin	Engineer	Viewer
Create organization	Yes	No	No	No
Manage billing	Yes	No	No	No
Manage integrations	Yes	Yes	No	No
Create projects/services	Yes	Yes	Yes	No
Edit service mappings	Yes	Yes	Yes	No
View dashboards	Yes	Yes	Yes	Yes
Download reports	Yes	Yes	Yes	Yes
Manage users	Yes	Yes	No	No

16. API Specification

Endpoint	Method	Purpose
/api/v1/orgs	POST/GET	Create/list organizations
/api/v1/orgs/{org_id}/projects	POST/GET	Create/list projects
/api/v1/projects/{project_id}/services	POST/GET	Create/list services
/api/v1/orgs/{org_id}/integrations	POST/GET	Configure/list integrations

Endpoint	Method	Purpose
/api/v1/integrations/{id}/test	POST	Connection and permission test
/api/v1/integrations/{id}/backfill	POST	Start backfill job
/api/v1/services/{id}/mappings	POST/GET	Manage service mappings
/api/v1/services/{id}/metrics/dora	GET	DORA metrics
/api/v1/services/{id}/metrics/reliability	GET	Reliability metrics
/api/v1/services/{id}/slos	POST/GET	SLO definitions
/api/v1/services/{id}/timeline	GET	Service Reliability Timeline
/api/v1/orgs/{id}/reports	POST/GET	Generate/list reports
/api/v1/webhooks/{provider}	POST	Provider webhook receiver

GET /api/v1/services/{service_id}/metrics/dora?range=30d&env=prod

Response:

```
{
  "service_id": "svc_123",
  "period": {"start": "2026-07-01", "end": "2026-07-31"},
  "deployment_frequency": {"value": 18, "unit": "deployments/month", "confidence": "high"},
  "lead_time": {"p50_hours": 18, "p75_hours": 42, "confidence": "medium"},
  "change_failure_rate": {"value_pct": 14.0, "confidence": "medium"},
  "failed_deployment_recovery_time": {"p50_minutes": 46, "confidence": "medium"},
  "deployment_rework_rate": {"value_pct": 8.0, "confidence": "low"}
}
```

17. Technology Stack

Layer	Recommended MVP Choice	Why
Frontend	Next.js + TypeScript	Fast dashboard development, modern SaaS UI, strong ecosystem
UI	Tailwind CSS + shadcn/ui	Professional components and fast iteration
Charts	Apache ECharts or Recharts	Flexible time-series and dashboard charts
Backend	FastAPI + Python 3.12	Excellent for APIs, integrations, data processing
ORM	SQLAlchemy 2 + Alembic	Mature Python data modeling and migrations
Workers	Celery or RQ + Redis	Backfills, sync jobs, aggregations, reports
Database	PostgreSQL 16	Strong relational + JSONB support
Time-series optional	TimescaleDB	Future scaling for metric snapshots/events
Object Storage	S3/MinIO	PDF reports and exports
Auth	Custom email/password + OAuth now; SAML later	MVP flexibility
IaC	Terraform	Repeatable AWS deployment
Container	Docker	Local/prod parity
CI/CD	GitHub Actions	Simple, low-cost MVP pipeline

17.1 Repository structure

```
bestdevops-pulse/
  apps/
    web/           # Next.js frontend
    api/           # FastAPI backend
    workers/       # Celery/RQ workers
  packages/
    shared-types/  # OpenAPI/generated types
    ui/            # reusable UI components
  infra/
    terraform/
    docker/
    k8s-later/
  docs/
```

```

architecture/
api/
runbooks/
scripts/
  seed_demo_data.py
  generate_report.py

```

18. DevOps and Infrastructure Plan

For development, a single Docker Compose stack is acceptable. For customer-facing production, use managed database and isolated secret storage from day one.

Environment	Architecture	Use Case
Local Dev	Docker Compose: web, api, worker, postgres, redis, minio	Developer productivity
Staging	Single EC2 or ECS + RDS dev instance	Pilot testing and demos
Production MVP	ECS Fargate + RDS PostgreSQL + ElastiCache + S3 + ALB + ACM	Customer pilots
Future Scale	EKS/ECS autoscaling + TimescaleDB + dedicated workers	Larger customers

18.1 CI/CD pipeline

Pull Request:

- lint frontend/backend
- type check
- unit tests
- build Docker images
- run API contract tests
- run DB migration dry-run

Merge to main:

- build and push images
- deploy to staging
- run smoke tests
- require approval for prod

Production deploy:

- run migrations
- rolling deploy API/web/workers
- verify health checks
- run synthetic checks

18.2 Backup and DR

Asset	Backup Requirement
PostgreSQL	Daily automated backup, PITR if using RDS, 30-day retention
S3 reports	Versioning and lifecycle rules
Secrets	KMS-managed, no plaintext backup
Terraform state	Remote encrypted backend
Restore test	Monthly restore drill for production

19. SRE Plan for BestDevOps Pulse

BestDevOps Service	SLI	MVP SLO
Web/API availability	Successful HTTP requests / total requests	99.5% monthly
API latency	p95 latency for dashboard API	p95 < 800ms
Ingestion freshness	Time since last successful sync per integration	95% integrations sync within 30 min

BestDevOps Service	SLI	MVP SLO
Report generation	Reports generated successfully	99% scheduled reports complete within 2h
Worker reliability	Successful jobs / total jobs	99% excluding external provider failures

Runbooks required for MVP:

- API down
- Worker queue backlog
- Integration sync failures
- Database high CPU or storage
- Webhook signature verification failures
- Report generation failure
- Customer data deletion request
- Credential rotation

20. QA and Testing Strategy

Test Type	MVP Coverage
Unit tests	Metric calculators, scoring, confidence, mapping rules
Integration tests	GitHub/GitLab/Jira/Jenkins/Prometheus connector mocks
Contract tests	External API payload fixtures and OpenAPI schema
E2E tests	Signup -> org -> integration -> project -> dashboard -> report
Data correctness tests	Known input events produce expected DORA/SRE/SLO outputs
Security tests	RBAC, tenant isolation, credential masking, webhook verification
Performance tests	Dashboard loads under 2s for 100 services and 12 months snapshots
Regression tests	Historical metric calculations do not drift unexpectedly

20.1 Golden datasets

Create deterministic fixture datasets for QA. These are more important than normal unit tests because metric correctness is the product.

Dataset	Purpose
Perfect Team	High deployment frequency, low failures, healthy SLO
Slow Team	Low deployment frequency and long lead time
Unstable Service	High change failure rate and repeated incidents
SLO Burn Service	Good delivery but budget burn high
Messy Data	Missing incident timestamps, manual deploys, incomplete mappings

21. Implementation Roadmap

Week	Focus	Deliverables
1	Foundation	Repo setup, local Docker, DB migrations, auth skeleton, design system
2	Tenancy model	Org/member/project/service CRUD, RBAC, audit log
3	Integration framework	Connector registry, credential vault, job queue, sync status
4	GitHub/GitLab MVP	Repo/PR/commit/workflow ingestion, external entities
5	Jira/manual incidents	Incident ingestion, manual CSV/form import, service mapping
6	CI/CD ingestion	GitHub Actions/Jenkins build/deployment normalization
7	DORA engine	Metric calculation, daily/weekly/monthly

Week	Focus	Deliverables
		snapshots, confidence
8	Reliability engine	Incident metrics, MTTA/MTTR, repeat incidents, postmortem tracking
9	SLO MVP	Manual SLO setup, Prometheus query integration, budget calculations
10	Dashboards	Org/project/service dashboards and filters
11	Timeline and reports	Service Reliability Timeline, PDF monthly report
12	Pilot hardening	Security review, QA, performance, onboarding docs, demo dataset

21.1 Team required

Role	MVP Allocation	Notes
Product Owner	0.5 FTE	Requirements, pilot feedback, prioritization
Tech Lead/Architect	1 FTE	Architecture, data model, reviews
Frontend Developer	1 FTE	Dashboards and onboarding
Backend Developer	1-2 FTE	API, data model, metric engines
Integration/Data Engineer	1 FTE	Connectors and normalization
QA Engineer	0.5-1 FTE	Golden datasets, automation, regression
DevOps/SRE	0.5 FTE	CI/CD, infra, monitoring, security

22. MVP Backlog

Epic 1 - Authentication and Tenancy

- Signup/login
- Organization creation
- Org roles
- Invitations
- Audit log

Epic 2 - Project/Service Model

- Project CRUD
- Service CRUD
- Environment config
- Criticality and owner
- Service mapping UI

Epic 3 - Integration Hub

- Provider catalog
- Credential storage
- Connection test
- Discovery jobs
- Sync status

Epic 4 - Data Ingestion

- GitHub/GitLab ingestion
- Jira/manual incident ingestion
- Jenkins/GitHub Actions ingestion

- Prometheus query ingestion

Epic 5 - Metric Engine

- DORA calculator
- Reliability calculator
- SLO calculator
- Snapshot scheduler
- Data confidence

Epic 6 - Dashboards

- Org dashboard
- Project dashboard
- Service dashboard
- Time filters
- Export CSV

Epic 7 - Timeline

- Timeline event model
- Timeline API
- Timeline UI
- Correlation rules

Epic 8 - Reports

- Monthly PDF template
- Report scheduler
- Report archive
- Download/share

Epic 9 - Security/Operations

- Tenant isolation tests
- Secret masking
- Webhook validation
- Observability
- Backup/runbooks

23. Risks and Mitigations

Risk	Impact	Mitigation
Metric definitions differ per company	Confusion and distrust	Allow configurable mapping rules and show definitions
Poor data quality	Wrong scores	Data Confidence Score and setup checklist
Service mapping is hard	Onboarding friction	Auto-suggest mappings but require user approval
External API limits	Backfill delays	Queue jobs, cursors, incremental sync, rate-limit strategy
Security concerns	Sales blocker	Read-only metadata, no source code, clear security docs
Competing DORA dashboards	Weak differentiation	Differentiate with service timeline, SLO correlation, monthly reports
Too much scope	MVP delay	Keep v1 limited to Delivery, Reliability, SLO

24. Launch and Pricing Plan

Offer	Price	Includes
Engineering Reliability Audit	\$999-\$4,999 one-time	Manual/hybrid setup, DORA + Reliability + SLO snapshot, PDF report, 90-day roadmap
Pulse Starter	\$499/month	1 org, 3 projects, 10 services, basic integrations, monthly report
Pulse Growth	\$1,999/month	1 org, 10 projects, 50 services, advanced integrations, historical reports
Pulse Business	\$4,999/month	Multiple teams, custom mappings, advanced SLO and support

Recommended go-to-market

Sell the Engineering Reliability Audit first. Use it to learn data patterns, build trust, and convert successful audits into the Pulse SaaS subscription.

25. Future Roadmap

Phase	New Capabilities
Phase 2	PagerDuty/Opsgenie, Datadog, OpenSLO import/export, ArgoCD, Slack alerts
Phase 3	DevSecOps score, SLSA/SBOM visibility, security posture integrations
Phase 4	Cost and waste intelligence: cloud, CI/CD, observability, SaaS license waste
Phase 5	Industry benchmarking using anonymized opt-in data
Phase 6	AI-assisted recommendation engine and executive summaries

26. Glossary

Term	Meaning
DORA	DevOps Research and Assessment metrics for software delivery performance
SLI	Service Level Indicator; quantitative reliability measure
SLO	Service Level Objective; target value/range for an SLI
Error Budget	Allowed unreliability for a service within an SLO window
Burn Rate	Speed at which error budget is being consumed
MTTA	Mean Time to Acknowledge
MTTR	Mean Time to Resolve/Recover
Service	A deployable application/component that can be measured independently
Mapping	Association between external tool resources and internal services
Data Confidence	Score indicating how reliable a metric calculation is based on source coverage and mapping quality

27. Reference Sources

[R1] DORA software delivery performance metrics: <https://dora.dev/guides/dora-metrics/>

[R2] Google SRE Book - Service Level Objectives: <https://sre.google/sre-book/service-level-objectives/>

[R3] Google SRE Workbook - Error Budget Policy: <https://sre.google/workbook/error-budget-policy/>

[R4] OpenSLO - open SLO language and YAML specification: <https://openslo.com/>

[R5] GitHub REST API permissions for fine-grained personal access tokens:

<https://docs.github.com/en/rest/authentication/permissions-required-for-fine-grained-personal-access-tokens>

[R6] Jira Software Cloud webhooks: <https://developer.atlassian.com/cloud/jira/software/webhooks/>

[R7] Jenkins Remote Access API: <https://www.jenkins.io/doc/book/using/remote-access-api/>

[R8] Prometheus HTTP API: <https://prometheus.io/docs/prometheus/latest/querying/api/>

[R9] Datadog Service Level Objectives API: <https://docs.datadoghq.com/api/latest/service-level-objectives/>

[R10] GitLab DORA metrics documentation: https://docs.gitlab.com/user/analytics/dora_metrics/

Appendix A - MVP Product Rules

MVP north star

A software leader should be able to open BestDevOps Pulse and know which services are healthy, which are risky, and what evidence supports that conclusion.

Rule	Decision	Reason
Service-first	Every core metric must be attributable to a service	Service-level data is actionable; project/org rollups are summaries only
Metadata-only	No source code or customer data required in MVP	Reduces security friction and sales objections
Read-only integrations	No external write actions in v1	Protects customer systems and simplifies approval
Explainable metrics	Every score must show raw metric and evidence	Prevents trust loss from black-box scoring
Confidence visible	Every metric shows data confidence	Engineering data is imperfect by nature
Rules before AI	Recommendations are rules-based first	Avoids hallucination and keeps output auditable
Dashboard + report	Both live dashboard and PDF report are required	CTOs share reports; engineers use dashboards

Appendix B - Detailed UX and Screen Specification

Screen	Primary User	Must Have Elements	Validation / Edge Cases
Signup/Login	All users	Email/password, Google login optional, terms checkbox	Duplicate email, unverified email, expired invite
Create Organization	Owner	Org name, slug, timezone, data region, company size	Slug uniqueness, timezone required
Invite Members	Owner/Admin	Email, role, invitation status	Invalid email, existing member, invite expiry
Integration Catalog	Admin	Cards for GitHub, GitLab, Jira, Jenkins, Prometheus, Datadog, Manual	Provider unavailable, missing permissions
Integration Setup	Admin	OAuth/token form, permissions explanation, test button	Failed auth, insufficient scope, token expiry
Discovery Results	Admin/Engineer	Repos, pipelines, Jira projects, SLO sources found	Large org pagination, partial discovery
Create Project	Admin/Engineer	Name, key, description, owner team	Duplicate key, archived project
Create Service	Admin/Engineer	Name, type, criticality, owner, environment aliases	Missing prod mapping, duplicate service key
Service Mapping	Admin/Engineer	External entity list, suggested mapping, confidence, manual override	Ambiguous repo names, unmapped incident components
Org Dashboard	All roles	Scorecards, trends, risky services, integration health, latest report	No data, low confidence, backfill running
Service Dashboard	All roles	DORA panel, reliability panel, SLO panel, confidence badges	No deployments, no SLOs, data gaps
Timeline	All roles	Chronological deploy/incident/SLO events, filters, evidence links	Duplicate webhooks, timezone rendering
Reports	All roles	Report list, generation status, download link	Failed generation, report too large

Onboarding checklist logic:
☐ Create organization

- [] Connect at least one code provider
- [] Connect or configure one deployment source
- [] Create at least one project
- [] Create or auto-discover at least one service
- [] Map repository + pipeline + production environment to service
- [] Add incidents manually or connect Jira
- [] Add at least one SLO manually or connect observability source
- [] Generate first dashboard snapshot
- [] Generate first PDF report

Appendix C - Detailed Data Dictionary

users

Column	Type	Notes
id	UUID	Primary key
email	text	Unique login email
name	text	Display name
status	enum	active/invited/disabled
created_at	timestampz	Creation time

organizations

Column	Type	Notes
id	UUID	Tenant id
name	text	Company/org name
slug	text	URL slug
timezone	text	Default timezone for reporting
plan	text	starter/growth/business
data_region	text	Future regional storage
created_by	UUID	Owner user id

projects

Column	Type	Notes
id	UUID	Project id
org_id	UUID	Tenant id
name	text	Project/product name
key	text	Short key
owner_team	text	Owning team
status	text	active/archived

services

Column	Type	Notes
id	UUID	Service id
org_id	UUID	Tenant id
project_id	UUID	Project id
name	text	Service name
service_key	text	Unique within org
type	enum	api/web/worker/mobile/library/infra
criticality	enum	low/medium/high/critical
owner_team	text	Team name

integrations

Column	Type	Notes
id	UUID	Integration id
org_id	UUID	Tenant id
provider	enum	github/gitlab/jira/jenkins/prometheus/datadog/manual
status	enum	configured/error/disabled
last_sync_at	timestampz	Last sync success
sync_cursor	jsonb	Provider cursor

external_entities

Column	Type	Notes
id	UUID	Entity id
integration_id	UUID	Source integration
entity_type	text	repo/pipeline/project/service/monitor
external_id	text	Provider id
name	text	Provider display name
metadata_json	jsonb	Provider metadata

service_mappings

Column	Type	Notes
id	UUID	Mapping id
service_id	UUID	BestDevOps service
external_entity_id	UUID	External entity
mapping_rule	text	manual/auto/name/label
confidence	int	0-100
created_by	UUID	User or system

deployments

Column	Type	Notes
id	UUID	Deployment id
service_id	UUID	Service
environment_id	UUID	Environment
version	text	Version/sha/tag
status	enum	success/failed/rollback/in_progress
started_at	timestampz	Start time
finished_at	timestampz	Finish time
source_url	text	Link to pipeline/deployment

incidents

Column	Type	Notes
id	UUID	Incident id
service_id	UUID	Service
severity	enum	p1/p2/p3/p4
status	enum	open/ack/resolved/postmortem_done
detected_at	timestampz	Detected/opened
acknowledged_at	timestampz	Ack time
resolved_at	timestampz	Resolve time
root_cause	text	Optional category
source_url	text	Jira/PagerDuty link

slo_definitions

Column	Type	Notes
id	UUID	SLO id
service_id	UUID	Service
name	text	SLO name
sli_type	enum	availability/latency/error_rate/custom
target	numeric	Example 99.9
window_days	int	Rolling window
query	text	PromQL/Datadog query or manual
status	text	active/disabled

metric_snapshots

Column	Type	Notes
id	UUID	Snapshot id
service_id	UUID	Service
metric_family	enum	dora/reliability/slo/pulse
granularity	enum	daily/weekly/monthly
period_start	date	Start
period_end	date	End
values_json	jsonb	Metric values
confidence_json	jsonb	Confidence values

Appendix D - Integration Permissions and Data Contracts

Provider	Minimum Access	Events / Data	Security Notes
GitHub	Read repository metadata, commits, pull requests, actions workflow runs	push, pull_request, workflow_run, deployment_status	Use GitHub App preferred; fine-grained permissions documented by GitHub [R5]
GitLab	Read projects, merge requests, commits, pipelines, deployments	push, merge_request, pipeline, deployment events	Support self-managed GitLab later
Jira	Read issues, projects, components, labels; webhook for issue changes	issue created/updated/resolved, labels/components	Jira webhook is HTTPS callback for events [R6]
Jenkins	Read jobs/builds/stages via API	job/build status and timestamps	Jenkins remote API returns JSON/XML/Python forms [R7]
Prometheus	Read query_range for SLI calculation	availability/error/latency series	Store only query results and query metadata; no raw logs [R8]
Datadog	Read SLOs and service monitor metadata	SLO status/history	Datadog has SLO API endpoints [R9]
Manual CSV	User upload	deployments/incidents/slo measurements	Validate schema before import; show manual confidence

Canonical event schema:

```
{
  "org_id": "uuid",
  "project_id": "uuid",
  "service_id": "uuid",
  "event_type": "deployment_succeeded|deployment_failed|incident_opened|incident_resolved|slo_burn|alert_fired",
  "event_time": "2026-07-01T10:14:00Z",
  "source_tool": "github_actions",
  "source_event_id": "provider-unique-id",
  "severity": "info|warning|critical",
  "payload": {"provider_specific": "redacted/minimized"}
}
```


Appendix E - Algorithm and Calculation LLD

E.1 Service mapping algorithm

```
def suggest_service_mapping(external_entity, services):
    candidates = []
    for service in services:
        score = 0
        if external_entity.name == service.service_key: score += 50
        if service.service_key in external_entity.name: score += 30
        if service.owner_team and service.owner_team in external_entity.metadata.get('team', ''): score += 10
        if external_entity.metadata.get('labels', []).contains(service.service_key): score += 20
        if external_entity.entity_type in expected_entity_types_for(service.type): score += 10
        candidates.append((service, min(score, 100)))
    return sorted(candidates, key=lambda x: x[1], reverse=True)[:5]
```

Rules:

- auto-map only if score >= 85 and no competing candidate within 10 points
- require human approval for score < 85
- all mapping changes create audit log

E.2 DORA calculation pseudocode

```
def calculate_dora(service_id, start, end, env='prod'):
    deployments = find_deployments(service_id, start, end, env, status_in=['success', 'failed', 'rollback'])
    successful = [d for d in deployments if d.status == 'success']
    failed = identify_failed_changes(service_id, deployments, start, end)

    deployment_frequency = count(successful)
    lead_times = []
    for d in successful:
        changes = find_changes_in_deployment(d)
        for c in changes:
            lead_times.append(d.finished_at - c.commit_created_at)

    cfr = count(failed) / max(count(deployments), 1)
    recovery_times = []
    for f in failed:
        incident = linked_incident_or_rollback(f)
        if incident and incident.resolved_at:
            recovery_times.append(incident.resolved_at - incident.detected_at)

    return {
        'deployment_frequency': deployment_frequency,
        'lead_time_p50': percentile(lead_times, 50),
        'lead_time_p75': percentile(lead_times, 75),
        'change_failure_rate': cfr,
        'recovery_time_p50': percentile(recovery_times, 50),
        'confidence': confidence_for_dora(service_id)
    }
```

E.3 Change failure identification rules

Rule	Signal	Confidence
Direct deployment failure	CI/CD deployment status failed	High
Rollback deployment	Deployment event status rollback or rollback tag	High
Incident shortly after deploy	Incident opened within configurable window after deployment for same service	Medium
Hotfix/revert	PR or commit label contains revert/hotfix/fix-forward	Medium
Manual incident correlation	User links incident to deployment	High
Unlabelled Jira bug	Bug ticket after deploy but no service mapping	Low

E.4 SLO burn-rate pseudocode

```
def calculate_error_budget(slo, start, end):
    target = slo.target / 100.0
    good, total = query_sli_events(slo, start, end)
    if total == 0:
        return no_data_result()
    actual_bad = total - good
    allowed_bad = total * (1 - target)
    consumed_pct = (actual_bad / allowed_bad) * 100 if allowed_bad > 0 else 100
    remaining_pct = max(0, 100 - consumed_pct)
    observed_error_rate = actual_bad / total
    allowed_error_rate = 1 - target
    burn_rate = observed_error_rate / allowed_error_rate if allowed_error_rate > 0 else 0
    return remaining_pct, burn_rate
```

Appendix F - API Payload Contracts

POST /api/v1/orgs

Request:

```
{
  "name": "Cotocus",
  "slug": "cotocus",
  "timezone": "Asia/Tokyo",
  "company_size": "51-200"
}
```

Response: 201 Created

```
{
  "id": "org_123",
  "name": "Cotocus",
  "slug": "cotocus",
  "setup_status": "integration_required"
}
```

POST /api/v1/projects/{project_id}/services

Request:

```
{
  "name": "Payment API",
  "service_key": "payment-api",
  "type": "api",
  "criticality": "critical",
  "owner_team": "Payments Platform",
  "production_aliases": ["prod", "production"]
}
```

POST /api/v1/services/{service_id}/slos

Request:

```
{
  "name": "Payment API Availability",
  "sli_type": "availability",
  "target": 99.9,
  "window_days": 28,
  "source": "prometheus",
  "good_query": "sum(rate(http_requests_total{service='payment-api', status!~'5..'}[5m]))",
  "total_query": "sum(rate(http_requests_total{service='payment-api'}[5m]))"
}
```

Appendix G - Frontend Component Specification

Component	Props / Inputs	Behavior
ScoreCard	title, score, trend, confidence	Shows score color, trend arrow, confidence badge
MetricTile	metric_name, value, unit, previous_value, confidence	Displays metric with percentage change
TimeRangeSelector	range, custom_start, custom_end	Controls all dashboard queries
ServiceRiskTable	services[]	Sortable by score, burn rate, MTTR, CFR

Component	Props / Inputs	Behavior
IntegrationHealthCard	provider, status, last_sync, errors	Shows setup and sync status
TimelineEventList	events[]	Chronological service story with filters
SloBudgetGauge	remaining_pct, burn_rate, target	Visualizes budget and risk state
ReportDownloadCard	report status, file_url	Generates/downloads PDF

Recommended frontend routes:

```

/app
/app/orgs/[orgSlug]
/app/orgs/[orgSlug]/projects
/app/orgs/[orgSlug]/projects/[projectKey]
/app/orgs/[orgSlug]/services/[serviceKey]
/app/orgs/[orgSlug]/services/[serviceKey]/timeline
/app/orgs/[orgSlug]/integrations
/app/orgs/[orgSlug]/mapping
/app/orgs/[orgSlug]/reports
/app/orgs/[orgSlug]/settings

```

Appendix H - Local Development Blueprint

Required tools:

- Docker Desktop or Rancher Desktop
- Node.js LTS
- Python 3.12
- Poetry or uv
- PostgreSQL client
- Make

Local services:

- web: Next.js dev server
- api: FastAPI uvicorn
- worker: Celery/RQ worker
- scheduler: Celery beat/APScheduler
- postgres: database
- redis: queue/cache
- minio: local object storage
- mailhog: local email testing

.env.example

```

APP_ENV=local
DATABASE_URL=postgresql+psycopg://pulse:pulse@postgres:5432/pulse
REDIS_URL=redis://redis:6379/0
OBJECT_STORAGE_ENDPOINT=http://minio:9000
OBJECT_STORAGE_BUCKET=reports
JWT_SECRET=change-me
ENCRYPTION_KEY=local-dev-key
GITHUB_CLIENT_ID=
GITHUB_CLIENT_SECRET=
JIRA_CLIENT_ID=
JIRA_CLIENT_SECRET=
PROMETHEUS_TEST_URL=http://prometheus:9090

```

Developer commands:

```

make setup          # install dependencies
make up             # start docker compose
make migrate        # run DB migrations
make seed           # seed demo org/project/services
make test           # all tests
make lint           # lint frontend/backend
make worker         # run worker locally
make report-demo    # generate sample PDF report

```

Appendix I - AWS Infrastructure Blueprint

Resource	MVP Choice	Notes
DNS	Route53	bestdevops.com and app.bestdevops.com
TLS	ACM	Managed certificate for HTTPS
Load Balancer	Application Load Balancer	Routes web/API traffic
Compute	ECS Fargate or EC2 Docker Compose for early pilot	Fargate preferred for production pilots
Database	RDS PostgreSQL	Automated backups and PITR
Queue/Cache	ElastiCache Redis or container Redis for first pilot	Managed recommended for production
Storage	S3	Reports, exports, encrypted objects
Secrets	AWS Secrets Manager + KMS	Integration credentials and app secrets
Logs	CloudWatch Logs	Centralized app/worker logs
Metrics	Prometheus/Grafana or CloudWatch	Operational visibility

Terraform modules:

```
infra/terraform/
  modules/
    network/
    ecs_service/
    rds_postgres/
    redis/
    s3_reports/
    secrets/
    observability/
  envs/
    dev/
    staging/
    prod/
```

Appendix J - CI/CD Blueprint

GitHub Actions workflow:

```
name: ci
on: [pull_request, push]
jobs:
  frontend:
    steps: checkout -> setup-node -> npm ci -> lint -> typecheck -> test -> build
  backend:
    steps: checkout -> setup-python -> install -> ruff -> mypy -> pytest
  docker:
    steps: build web/api/worker images
  migrations:
    steps: alembic check/dry-run
  security:
    steps: dependency scan, secret scan, container scan
```

Deploy workflow:

- triggered on main merge or manual dispatch
- pushes images to ECR
- runs migrations
- deploys ECS services
- runs smoke tests
- sends deployment event to BestDevOps itself later

Appendix K - QA Test Case Matrix

ID	Scenario	Expected Result
QA-001	Create org with valid data	Org created and setup checklist visible
QA-002	Viewer attempts integration setup	Access denied and audit event created
QA-003	GitHub token missing repo read permission	Connection test fails with clear scope

ID	Scenario	Expected Result
		message
QA-004	Backfill 180 days GitHub data	Job completes with cursor and discovered repos
QA-005	Auto-map repo payment-api to Payment API service	Suggestion confidence >= 85
QA-006	Ambiguous repo mapping	System requires manual approval
QA-007	Known deployment fixture with 10 deploys	Deployment frequency = 10 for period
QA-008	Known incident fixture with open/ack/resolve times	MTTA/MTTR match expected values
QA-009	SLO good=999000 total=1000000 target=99.9	Budget remaining = 0% when bad=1000 consumed=100%
QA-010	Metric with no CI data	Confidence shown as low, not blank score
QA-011	Timeline with duplicate webhook	Duplicate is ignored by unique source_event_id
QA-012	Monthly report generation	PDF file created and report record stored
QA-013	Tenant isolation	User from Org A cannot access Org B data
QA-014	Webhook invalid signature	Rejected and logged without processing
QA-015	Dashboard 100 services, 12 months snapshots	Page loads under target threshold

Appendix L - Operations Runbooks

API down

25. Check ALB target health
26. Check ECS/EC2 service status
27. Check DB connectivity
28. Rollback latest deploy if needed
29. Create incident and update status page later

Worker backlog

30. Check Redis queue size
31. Scale worker count
32. Identify failing job type
33. Pause problematic integration if external provider issue
34. Re-run failed jobs after fix

Integration sync failing

35. Check provider API status
36. Check token expiry
37. Run permission test
38. Inspect last error payload
39. Notify org admin if credential action needed

Bad metric detected

40. Identify affected service and period
41. Check source events
42. Check mappings
43. Mark metric snapshot stale
44. Recalculate after correction
45. Add regression test fixture

Report generation failure

46. Check report worker logs
47. Validate data availability
48. Retry generation
49. Fallback to HTML report if PDF renderer fails

Security incident

50. Disable affected credential
51. Rotate keys
52. Review audit logs
53. Notify affected customers if applicable
54. Preserve evidence

Appendix M - Pilot Customer Implementation Checklist

Phase	Checklist
Pre-sales	Identify 3-10 services; confirm tools used; confirm no source code access required; agree success criteria
Setup Day 1	Create org; invite admin; connect GitHub/GitLab; connect Jira/manual incidents; create projects/services
Setup Day 2	Map repos/pipelines/incidents/SLOs to services; run backfill; review data confidence
Week 1	Validate first DORA and reliability metrics with customer team
Week 2	Enable SLO/error budget for at least 2 critical services
Week 3	Generate first report; review insights; collect feedback
Week 4	Convert to monthly subscription or define missing integration blockers

Pilot success criteria

The customer agrees that BestDevOps Pulse connected delivery and reliability evidence better than their existing tools and wants continuous monthly tracking.

Appendix N - Known Limitations of MVP

Limitation	Reason	Planned Fix
Change failure rate may be approximate	Requires incident/deployment correlation	Add better PagerDuty/Opsgenie integrations and manual linking
Lead time may vary by workflow	Some teams squash commits or deploy batches	Allow PR-merge-based and commit-based calculation modes
SLO automation may be limited	Observability data retention varies	Support OpenSLO, Datadog, Grafana later
Manual imports reduce confidence	CSV data may be incomplete	Show confidence and encourage integrations
No advanced enterprise SSO	Not needed for first pilots	Add SAML/SCIM in business/enterprise plan
No cloud cost or AI readiness	Out of v1 scope	Future modules after core product proves demand

Appendix O - Final MVP Build Checklist

- Auth and org creation works end-to-end

- RBAC blocks unauthorized actions
- Project/service model is stable
- At least one code provider integration works
- At least one CI/CD source works
- At least one incident source/manual import works
- At least one SLO source/manual mode works
- Metric snapshots are calculated daily/weekly/monthly
- Dashboard supports week/month/3m/6m/1y/custom filters
- Service Reliability Timeline shows correlated events
- Data Confidence Score is visible
- Monthly PDF report is generated
- Tenant isolation tests pass
- Credential encryption and masking implemented
- Backup/restore tested
- Pilot onboarding guide written
- Demo dataset available
- Security promise documented: no source code required